Promises_explained_in_detail

18/08/2026

Source: https://chatgpt.com/?prompt=Why+here+the+handler+one+is+not+getting+removed%3F+It+is+still+printing+in+console.log+in+1%27%29%0A%7D%0A%0Alet+handler2+%3D+%28e%29+%3D%3E+%7B%0A++++console.log%28%2+2%27%29%0A%7D%0A%0Alet+handler3+%3D+%28e%29+%3D%3E+%7B%0A++++console.log%28%27HA+3%27%29%0A%7D%0A%0Alet+handler4+%3D+%28e%29+%3D%3E+%7B%0A++++console.log%28%27HA+4%27%29%0A%7D%0A%0A%2F%2F+create+ele%0Alet+button3+%3D+document.createElement%28%27b&chat=true

1. I'm learning promises in JavaScript and currently the tutorial I'm following showed a very simple way of, you know, reading promise. But there is one, you know, doubt I have. I need to make sure like what I am thinking is right or wrong. This is the code: let promise = new Promise((resolve , reject) => { console.log('i am a promise') resolve('sucess', 123) }) let promise2 = new Promise((resolve , reject) => { console.log('i am a promise - 2') reject('failed - some error occured', 400) }) So like this promises work like this, basically, it is looking to me like an object and, uh, created with new keyword, and inside the promise, like we pass an arrow function and inside that arrow function, there is some task that ne. 3
2. See, the tutorial I am following, that person, teacher also said that usually we don't create promises because we fire an API request and we handle promises, we don't create. But for the learning purpose, I am also learning like how promises work and all. So my question was, how does a code block inside a promise executes and how does, like, when to return resolve or reject is determined and like inside the same block how it is. Like the example he showed me here, whatever the code block is written, it will return, reject or resolve whatever is written. But then how it is handled that when successful, then return this, when unsuccessful, return this, return the reject statement how?Show moreShow less. 8
3. let num = 4; let promise3 = new Promise((resolve, reject) => { let number = num; if (number > 5) { resolve({ message : "Sucess", statusCode : 201 }); } else { reject({ message : "Failed : Some error occured", statusCode : 404 }); } }); // this is how we usually deal with promises inside the code base let promise4 = new Promise((resolve, reject) => { let age = 20; if (age >= 18) { resolve("You are allowed"); } else { reject("You are not allowed"); } }); promise4 .then((result) => { console.log(result); }) .catch((error) => { console.log(error); }); but the output: You are a. 14
4. Okay, I understood this part. But see, my real question is, I still have one small question in my mind. Like, I can now understand how we deal with promises whenever we, you know, use .then, .catch in the fetch URL thing to, you know, handle it, whatever the response is coming. But again, my, still I have a question, that is, how do APIs handle it? Let's say, don't think so, like, I am, you know, engineer who is requesting an API and getting a response or error in that way. But how things are working in the API itself? do they also use conditionals to determine resolve and rejectShow moreShow less. 20
5. Okay, so I will explain all of the things that I have understood so far or the doubts I have to consolidate everything, and you tell me like what I'm saying is right or wrong or if there is any changes, corrections you want to make. So the first thing is promises themselves are objects. It is not like we are directly making promises in the code or something like that, but I'm learning this so that I can understand the fetch properly. Now, fetch keyword returns a promise itself, so I need to handle that promise. Okay? So, as far as I can understood that this thing, fetch request fires a query, like, not like fetch request fires a query. If a query is fired from, let's suppose, front-end, I think fetch keyword itself also fires it. So once it. 25
6. Now there are few questions of mine. Like you said that when the server sends 404, the promise is fulfilled, that is considered a valid response, and our fetch keyword receives it, and then returns a promise. Then we need to handle it with .then and .catch. And you are saying that the response object has a property called .ok. So, ok is true for status code like 200 and false for 400. But, you know, like there are series of status codes like 100 and so on, 200 so on, 300 so on, 400 so on, 500 so on. So like, who is defining all of these, like, whether 200 is okay while 400 is not? And then you told me that now based on the response that you received from fetch, then you can apply .then to convert that response according to our need. For ex. 36

7. Okay, now I have understood almost all the things. Just two things I want to know more. That is, first, you said that, you know, the status code and all that comes, that's not a part of JavaScript, but the HTTP header itself. And it is defined that whether to consider this range of status code as request.ok true or false. Okay? So, and I think there might be some stored values for every range from 100 to 199, 200 to 299, till 500 and so on. So, like, my question is, does that mean like response.ok, so basically it is happening something like this. You tell me whether I am right or wrong. So basically, response is an object, first of all. And it has a property named ok. And ok stores a value like we have an object key-value pair. So ok here. 47
8. Okay, now just one last request, like one more point I need to be clarified in my mind. That is the production side of promises. Like, now I'm not talking about handling the API response, but how APIs themselves are created. I think promises are used over there. So now my question is, how resolve and reject are used over there? Like, you said that in the HTTP request, there are many metadata that comes as a response from the server. So like all those responses are bundled inside the reject and resolve, and then passed as a response here, or what? Like, and if possible, give me one example of such code.Show moreShow less. 56
9. Okay, so now just do one thing. All the doubts that I had, all the points that you explained me so far regarding promises and all, everything, summarize it. All of them. Put all of the points in brief so that all of them can fit in - and summarize all of the thing that we have studied above in detail don’t miss any single pointShow moreShow less. 63

1. I'm learning promises in JavaScript and currently the tutorial I'm following showed a very simple way of, you know, reading promise. But there is one, you know, doubt I have. I need to make sure like what I am thinking is right or wrong. This is the code:

   ```
   let promise = new Promise((resolve , reject) => {

       console.log('i am a promise')

       resolve('sucess', 123)

   })

   let promise2 = new Promise((resolve , reject) => {

       console.log('i am a promise - 2')

       reject('failed - some error occured', 400)

   })
   ```

   So like this promises work like this, basically, it is looking to me like an object and, uh, created with new keyword, and inside the promise, like we pass an arrow function and inside that arrow function, there is some task that need to be done. And once the task is done, if it is done successfully, it gives the message of whatever written inside the resolve. But if it is failed, then it returns the message of whatever is written inside reject.

   Show moreShow less

Yes — **your understanding is mostly correct**, and you're actually noticing an important detail that your tutorial may not have explained yet.

Let's break your code down.

1. A Promise is an object

Yes:

```JavaScript
let promise = new Promise((resolve, reject) => {
    // ...
})
```

`new Promise(...)` creates a **Promise object**.

The `Promise` constructor takes a function as its argument. That function is called the **executor function**:

```JavaScript
```

```
(resolve, reject) => {
    // executor
}
```

And JavaScript gives that function two functions:

- **`resolve`**
- **`reject`**

You don't create those two functions yourself.

---

2. The executor runs immediately

This is an important point.

When you do:

```JavaScript
let promise = new Promise((resolve, reject) => {
    console.log("I am a promise");
    resolve("success");
});
```

the code inside the executor runs **immediately**.

So this:

```JavaScript
console.log("I am a promise");
```

prints immediately when the Promise is created.

A Promise does **not** automatically mean "run this code later."

Usually, the work inside a Promise represents something asynchronous, such as:

```JavaScript
fetch(...)
```

a timer, reading a file, database operation, etc.

---

3. `resolve()` means the operation succeeded

You have this:

```JavaScript
resolve('sucess', 123)
```

Conceptually, you're thinking:

"The operation succeeded, so I'll return `'success'` and `123`."

There's one small correction here:

`resolve()` only accepts one value.

So:

```JavaScript
resolve('success', 123)
```

is effectively the same as:

```JavaScript
resolve('success')
```

The `123` is ignored.

If you want to resolve with multiple pieces of information, you could do:

```JavaScript
resolve({
    message: 'success',
    number: 123
});
```

Then the resolved value is the entire object.

---

4. `reject()` is the same idea

You have:

```JavaScript
reject('failed - some error occured', 400)
```

Again, only the **first argument** matters.

So this:

```JavaScript
reject('failed - some error occured', 400)
```

is effectively:

```JavaScript
reject('failed - some error occured')
```

You could instead do:

```JavaScript
```

```
reject({
    message: 'failed - some error occured',
    code: 400
});
```

5. One really important distinction

You said:

> "once the task is done, if it is done successfully, it gives the message of whatever written inside the resolve. But if it is failed, then it returns the message of whatever is written inside reject."

That's **very close**, but I'd change the wording slightly.

`resolve()` and **`reject()`** don't directly "return" the value to you.

They **change the state of the Promise** and provide a value that you can later consume with **`.then()`** and **`.catch()`**.

For example:

```
JavaScript
let promise = new Promise((resolve, reject) => {
    resolve("success");
});

promise.then((value) => {
    console.log(value);
});
```

Output:

```
success
```

And:

```
JavaScript
let promise2 = new Promise((resolve, reject) => {
    reject("failed");
});

promise2.catch((error) => {
    console.log(error);
});
```

Output:

```
failed
```

So you can think of it like:

```
                Promise
                   |
            +------+------+
            |             |
```

```
resolve        reject
   |             |
success        failure
   |             |
.then()        .catch()
```

One more thing about your exact code

You currently have:

```JavaScript
let promise = new Promise((resolve, reject) => {
    console.log('i am a promise')
    resolve('sucess', 123)
})

let promise2 = new Promise((resolve, reject) => {
    console.log('i am a promise - 2')
    reject('failed - some error occured', 400)
})
```

Both executors run immediately.

So just running this code prints:

```
i am a promise
i am a promise - 2
```

But **you won't see `"success"` or `"failed"` printed**, because you haven't consumed the Promise yet.

You would need something like:

```JavaScript
promise.then((value) => {
    console.log(value);
});

promise2.catch((error) => {
    console.log(error);
});
```

Then you'd see:

```
i am a promise
i am a promise - 2
success
failed - some error occured
```

So your mental model is basically right:

> **A Promise represents the eventual result of an operation. `resolve(value)` fulfills the Promise with a value, while `reject(reason)` rejects it with a reason. `.then()` is used to handle fulfillment, and `.catch()` is used to handle rejection.**

And one terminology tweak: don't think of **`resolve`/`reject`** as "returning" something. Think of them as **settling the Promise**. That's the terminology you'll see a lot as you go deeper.

2. See, the tutorial I am following, that person, teacher also said that usually we don't create promises because we fire an API request and we handle promises, we don't create. But for the learning purpose, I am also learning like how promises work and all. So my question was, how does a code block inside a promise executes and how does, like, when to return resolve or reject is determined and like inside the same block how it is. Like the example he showed me here, whatever the code block is written, it will return, reject or resolve whatever is written. But then how it is handled that when successful, then return this, when unsuccessful, return this, return the reject statement how?
Show moreShow less

Exactly — **this is the key question about Promises**. The confusing part is that the Promise itself does **not** know whether your operation was successful or unsuccessful.

You, or the API/library you're using, decide that by calling either `resolve()` or `reject()`.

Let's build the mental model from your example.

Think of `resolve` and `reject` as two buttons

When you write:

```JavaScript
let promise = new Promise((resolve, reject) => {

    // do some work...

});
```

JavaScript essentially gives your function two special functions:

```
resolve → "The operation succeeded!"
reject  → "The operation failed!"
```

But **JavaScript doesn't automatically inspect your code and decide which one to call.**

Your code has to decide.

For example:

```JavaScript
let promise = new Promise((resolve, reject) => {

    let number = 10;

    if (number > 5) {
        resolve("It worked!");
    } else {
        reject("It failed!");
    }

});
```

Here **you** have written the condition that determines what happens.

If `number > 5` is true:

```JavaScript
resolve("It worked!");
```

gets executed.

If it's false:

```JavaScript
reject("It failed!");
```

gets executed.

But then how does this work with an API?

This is where Promises become much more useful.

Imagine a simplified version of an API request:

```JavaScript
let promise = new Promise((resolve, reject) => {

    // make API request

    if (API_request_was_successful) {
        resolve(data);
    } else {
        reject(error);
    }

});
```

Notice something important:

The API request itself determines whether we call `resolve` or `reject`.

For example, imagine you're requesting:

```
GET /users
```

The server might respond:

```
200 OK
```

So the code handling the request says:

```JavaScript
resolve(data);
```

But if the request fails:

```
500 Internal Server Error
```

then the code says:

```JavaScript
reject(error);
```

So the Promise isn't magically determining success.

It's more like:

> **"I'm going to wait for somebody to tell me whether this operation succeeded or failed."**

And this explains why your simple example feels strange

Your example is:

```JavaScript
let promise = new Promise((resolve, reject) => {
    console.log('i am a promise');
    resolve('success', 123);
});
```

There isn't actually any operation here that could fail.

So you're simply saying:

```JavaScript
resolve('success');
```

which means:

> "Immediately consider this Promise successful."

And this:

```JavaScript
let promise2 = new Promise((resolve, reject) => {
    console.log('i am a promise - 2');
    reject('failed - some error occured', 400);
});
```

means:

> "Immediately consider this Promise failed."

So the tutorial is probably using these examples **only to demonstrate how the Promise mechanism works**, not to demonstrate a realistic success/failure decision.

Here's a more realistic example

Suppose we're checking someone's age:

```JavaScript
let promise = new Promise((resolve, reject) => {

    let age = 20;

    if (age >= 18) {
        resolve("You are allowed");
    } else {
        reject("You are not allowed");
    }

});
```

Now we have an actual decision.

If:

```JavaScript
age = 20
```

then:

```JavaScript
resolve("You are allowed");
```

runs.

The Promise becomes:

```
FULFILLED
```

If:

```JavaScript
age = 15
```

then:

```JavaScript
reject("You are not allowed");
```

runs.

The Promise becomes:

```
REJECTED
```

And **then** something else can handle that result:

```JavaScript
promise
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    });
```

Now connect this to an API

This is the really important part for your learning.

You might eventually write something like:

```JavaScript
fetch("https://example.com/users")
    .then((response) => {
        console.log(response);
    })
    .catch((error) => {
        console.log(error);
    });
```

You're thinking:

> "Wait, where did **`resolve()`** and **`reject()`** go?"

They're **inside the implementation of `fetch()`**.

You don't normally see them.

Conceptually, you can imagine **`fetch()`** doing something like:

```JavaScript
function myFetch(url) {

    return new Promise((resolve, reject) => {

        // make network request...

        if (requestSucceeded) {
            resolve(response);
        } else {
            reject(error);
        }

    });

}
```

That's a simplified mental model, not the actual implementation.

So when you write:

```JavaScript
fetch(url)
```

`fetch()` creates and returns a Promise.

Internally, once the network operation finishes, the implementation settles that Promise.

Then your:

```JavaScript
.then(...)
```

handles the successful result.

And:

```JavaScript
.catch(...)
```

handles the rejection.

So there are actually two separate things happening

This distinction will make Promises much easier for you:

Part 1 — Creating/settling the Promise

Some code performs an operation:

```JavaScript
new Promise((resolve, reject) => {

    // do something

    if (success) {
        resolve(result);
    } else {
        reject(error);
    }

});
```

Part 2 — Consuming the Promise

Other code says:

```JavaScript
promise
    .then(result => {
        // what should I do with success?
    })
    .catch(error => {
```

```
    // what should I do with failure?
});
```

So:

```
      SOME OPERATION
            |
       Did it finish?
       /           \
    YES             NO
     |               |
  resolve          reject
     |               |
     v               v
  Promise         Promise
fulfilled         rejected
     |               |
     v               v
  .then()         .catch()
```

The biggest takeaway

The Promise does not decide whether something is successful.

The code performing the operation decides.

For your learning examples, **you manually decide**:

```
JavaScript
resolve(...)
```

or:

```
JavaScript
reject(...)
```

For real-world APIs, the **API/request mechanism decides** based on what happens during the operation.

And that's exactly why your teacher said that in real development you usually **consume promises rather than manually create them**. Libraries like `fetch()` create the Promise for you; you mostly deal with the `.then()`, `.catch()`, or `await` side.

```
3. let num = 4;

let promise3 = new Promise((resolve, reject) => {
    let number = num;

    if (number > 5) {
        resolve({
            message : "Sucess",
            statusCode : 201
        });
    } else {
```

```
        reject({
            message : "Failed : Some error occured",
            statusCode : 404
        });
    }
});
// this is how we usually deal with promises inside the code base

let promise4 = new Promise((resolve, reject) => {

    let age = 20;

    if (age >= 18) {
        resolve("You are allowed");
    } else {
        reject("You are not allowed");
    }

});

promise4
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    });

but the output:


You are allowed
promisesAndPromiseChainScript.js:59 Uncaught (in promise) #<Object>
(anonymous) @ promisesAndPromiseChainScript.js:59
(anonymous) @ promisesAndPromiseChainScript.js:50
Show moreShow less
```

Yes — and this output is actually showing you **two different Promises behaving independently**. Your **`promise4`** is working correctly. The error is coming from **`promise3`**.

Let's trace it carefully.

`promise3`

You have:

```JavaScript
let num = 4;
```

```
let promise3 = new Promise((resolve, reject) => {
    let number = num;

    if (number > 5) {
        resolve({
            message: "Sucess",
            statusCode: 201
        });
    } else {
        reject({
            message: "Failed : Some error occured",
            statusCode: 404
        });
    }
});
```

num is **4**.

Therefore:

```
JavaScript
number > 5
```

is:

```
JavaScript
4 > 5   // false
```

So this runs:

```
JavaScript
reject({
    message: "Failed : Some error occured",
    statusCode: 404
});
```

Therefore **promise3** becomes:

```
REJECTED
```

And importantly, the rejection value is an **object**:

```
JavaScript
{
    message: "Failed : Some error occured",
    statusCode: 404
}
```

But here's the problem

You never handle **promise3**.

You created it:

```JavaScript
let promise3 = new Promise(...)
```

but you never did:

```JavaScript
promise3.catch(...)
```

or:

```JavaScript
promise3.then(...).catch(...)
```

So the browser is essentially saying:

> "Hey, you created a Promise, it was rejected, and nobody handled that rejection."

That's why you're seeing:

```
Uncaught (in promise) #<Object>
```

The **`#<Object>`** is because you rejected the Promise with an **object**, rather than a string.

Meanwhile, `promise4` is completely fine

This:

```JavaScript
let age = 20;

if (age >= 18) {
    resolve("You are allowed");
}
```

means:

```
20 >= 18
       ↓
     true
       ↓
   resolve()
       ↓
Promise fulfilled
```

And you correctly handle it:

```JavaScript
promise4
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
```

```
        console.log(error);
    });
```

So you get:

```
You are allowed
```

That's why you're seeing that output.

Try handling `promise3`

For example:

```JavaScript
promise3
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    });
```

Now you'll get something like:

```
{
    message: "Failed : Some error occured",
    statusCode: 404
}
```

And if you want to access the individual properties:

```JavaScript
promise3
    .then((result) => {
        console.log(result.message);
        console.log(result.statusCode);
    })
    .catch((error) => {
        console.log(error.message);
        console.log(error.statusCode);
    });
```

Output:

```
Failed : Some error occured
404
```

And this is actually a very important lesson

Look at the two Promises:

```
JavaScript
promise3 → rejected
promise4 → fulfilled
```

They are **separate objects**.

Creating **`promise4`** doesn't somehow handle **`promise3`**.

You have to explicitly attach a handler to each Promise that you care about:

```
JavaScript
promise3.catch(...);

promise4.then(...).catch(...);
```

So your current code is essentially doing this:

```
promise3
   |
   ↓
REJECTED
   |
   ↓
Nobody handles it ❌
   |
   ↓
Uncaught (in promise)


promise4
   |
   ↓
FULFILLED
   |
   ↓
.then()
   |
   ↓
"You are allowed" ✅
```

One small improvement

In real JavaScript code, it's generally better to reject with an **`Error`** object rather than an arbitrary object:

```
JavaScript
reject(new Error("Some error occurred"));
```

Then:

```
JavaScript
promise3.catch((error) => {
    console.log(error.message);
});
```

But **for learning how Promise values work**, your object is perfectly fine and is actually useful for understanding that **`resolve()`** and **`reject()`** can carry values.

And your understanding is progressing correctly: **`resolve()`/`reject()` settle the Promise; `.then()`/`.catch()` are how other code reacts to that settled result.**

4. Okay, I understood this part. But see, my real question is, I still have one small question in my mind. Like, I can now understand how we deal with promises whenever we, you know, use .then, .catch in the fetch URL thing to, you know, handle it, whatever the response is coming. But again, my, still I have a question, that is, how do APIs handle it? Let's say, don't think so, like, I am, you know, engineer who is requesting an API and getting a response or error in that way. But how things are working in the API itself? do they also use conditionals to determine resolve and reject Show moreShow less

Yes — **you've reached the exact point where Promises, HTTP, and backend APIs can get mixed together.** The answer is:

> **The API server itself usually does not use `resolve()` and `reject()` to send an HTTP response to you.**

There are actually **two different layers** here.

1. Your JavaScript application

When you do:

```JavaScript
fetch("/users")
```

`fetch()` returns a Promise.

Inside the implementation of **`fetch()`**, there is asynchronous/networking code that eventually settles that Promise based on what happens with the request.

Conceptually, you can imagine:

```JavaScript
new Promise((resolve, reject) => {

    // send HTTP request...

    if (networkRequestSucceeded) {
        resolve(response);
    } else {
        reject(error);
    }

});
```

That's roughly the layer you're learning about.

---

2. The actual API server

Suppose your browser sends:

```
GET /users
```

to a backend server.

The backend might be written in Node.js, Java, Python, Go, C#, etc.

The backend does something like:

```
Receive request
      ↓
Check authentication
      ↓
Query database
      ↓
Did database operation succeed?
      ↓
   ┌──YES──┐
   ↓       ↓
Return    Return
200       500
+ data   + error
```

It might indeed use **conditionals**:

```JavaScript
if (userExists) {
    // send successful HTTP response
} else {
    // send error HTTP response
}
```

But that's **not the same thing as `resolve()` and `reject()`.**

Here's the important distinction

Imagine you're building an API with Node.js.

You might have something conceptually like:

```JavaScript
app.get("/users/:id", async (req, res) => {

    const user = await database.findUser(req.params.id);

    if (user) {
        res.status(200).json(user);
    } else {
        res.status(404).json({
            message: "User not found"
        });
    }

});
```

Here the backend is deciding:

```
User found?
   │
   ├── YES → HTTP 200 + user data
   │
   └── NO  → HTTP 404 + error information
```

Notice we're not doing:

```JavaScript
resolve(...)
reject(...)
```

We're sending an **HTTP response**.

Then where do `resolve()` and `reject()` come into the picture?

Let's connect the whole journey.

Suppose you do this in your frontend:

```JavaScript
fetch("/users/123")
    .then(response => response.json())
    .then(data => {
        console.log(data);
    })
    .catch(error => {
        console.log(error);
    });
```

The journey is roughly:

```
YOUR FRONTEND
     │
     │ HTTP request
     ↓
   SERVER
     │
     │ checks things
     │ queries database
     │
     ↓
HTTP RESPONSE
     │
     │ 200 / 404 / 500 / etc.
     ↓
YOUR BROWSER
     │
     ↓
fetch() settles its Promise
     │
     ├── fulfilled → .then()
     │
     └── rejected  → .catch()
```

So there are **two separate decisions** happening.

Server decision

The server decides what HTTP response to send:

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
404 Not Found
500 Internal Server Error
...
```

Client/`fetch()` decision

The browser's **`fetch()`** machinery deals with the network operation and its Promise.

And here's a **very important detail**:

A **`404`** or **`500`** HTTP response does **not normally make `fetch()` reject its Promise**.

For example:

```JavaScript
fetch("/users/123")
    .then(response => {
        console.log(response.status);
    })
    .catch(error => {
        console.log(error);
    });
```

If the server returns:

```
404 Not Found
```

`fetch()` normally still fulfills its Promise with a **`Response`** object.

So you'll typically get:

```
404
```

inside **`.then()`**.

`catch()` is generally for things such as a network failure where a response wasn't successfully obtained.

If you want HTTP errors like 404/500 to enter your own error handling, you commonly do:

```JavaScript
fetch("/users/123")
    .then(response => {

        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }

        return response.json();
    })
    .then(data => {
        console.log(data);
    })
    .catch(error => {
```

```
        console.log(error);
    });
```

Now **you** are turning an HTTP error into a rejected Promise.

And what about the backend's database calls?

This is where your original question becomes even more interesting.

The backend itself may also use Promises!

For example, Node.js code might do:

```JavaScript
const user = await database.findUser(id);
```

Under the hood, the database library may have created a Promise for that database operation.

So there can be **Promises at multiple levels**:

```
Frontend
   |
   | fetch() Promise
   ↓
Internet
   |
   ↓
Backend
   |
   | database query Promise
   ↓
Database
```

The backend could have:

```JavaScript
try {
    const user = await database.findUser(id);

    if (user) {
        res.status(200).json(user);
    } else {
        res.status(404).json({ message: "Not found" });
    }

} catch (error) {
    res.status(500).json({ message: "Database error" });
}
```

Here:

- **`database.findUser()`** may return a Promise.
- **`await`** waits for that Promise.
- The backend then decides what **HTTP response** to send.
- The frontend's **`fetch()`** handles the network operation and gives you its own Promise.

So **Promises aren't an "API thing."** They're a JavaScript mechanism for representing an eventual result of asynchronous work.

HTTP status codes are a **different mechanism** for communicating the result of an HTTP request.

That's the conceptual separation I'd recommend keeping in your head:

> **Promise:** "When this asynchronous operation finishes, here's how we'll represent its eventual result."
>
> **HTTP response:** "The server is telling the client what happened with this HTTP request."

Once you separate those two ideas, the whole **`fetch → API → backend → database → Promise`** chain becomes much easier to understand.

5. Okay, so I will explain all of the things that I have understood so far or the doubts I have to consolidate everything, and you tell me like what I'm saying is right or wrong or if there is any changes, corrections you want to make. So the first thing is promises themselves are objects. It is not like we are directly making promises in the code or something like that, but I'm learning this so that I can understand the fetch properly. Now, fetch keyword returns a promise itself, so I need to handle that promise. Okay? So, as far as I can understood that this thing, fetch request fires a query, like, not like fetch request fires a query. If a query is fired from, let's suppose, front-end, I think fetch keyword itself also fires it. So once it fires the query, after going through all the networks, it reaches the server. Now, server receives that query or, you know, request. Then on server side, there is no such thing as internally promises and all, but rather it deals with two parameters, that is request and response. So what server does is, it receives the request and queries, and make the queries for that request. If the request, if it finds it, it gives the response back along with status code, let's say 200. Or if not, then it will give the response back with status code 400 and a message, like, not found, in the same JSON format. So both of these things are responses. There is no such thing as reject or resolve here right now. Now, along the way, when it comes from the server back again to our, you know, client-side device, there are many network layers. So the catch plays a role here that if anything mishappens between the network and I'm not able to receive both the response and status code properly, so in between the network, anything that, you know, that is happening, that particular node or level will fire, give a response of their own, that will be stored in, you know, the catch as an error or in the reject parameter. And if not, then everything internally is considered fulfilled only. There is no such thing as rejection. All the responses from the server is, you know, received by us in .then method. Basically, it is all considered as resolved only.

   Now the few doubts that I have is, first, you know, inside the .then and .catch method, I'm not able to understand. First of all, like, you are writing an arrow function with, you know, a parameter response. So, is this response a keyword or what? Secondly, same goes for, you know, catch, the error and all. Like, please explain them to me. The third is the data. I can see that you are writing, you know, in response, in .then method, data arrow function with parameter data. So, like, what is this? We haven't defined data or anything like that. So where does this data come from? Is this also a keyword or what? And my next question is, I understood like .catch, catches any error which happened in between the network, if it is happening. But again, it also catches the, you know, false response or something like that coming directly from the server as well, if the query was not

fulfilled. At that point, at server side. The promise is returning is considered fulfilled because it is giving the response. But in the response itself, if it is giving, you know, an error that it doesn't found the data on the server or database. Then how catch is able to, you know, intercept that as well and print it as error.
Show moreShow less

Yes. You're **very close**, and your overall mental model is now solid. There are just **two important corrections** to make before we get to your questions.

First, corrections to your understanding

✅ 1. `fetch()` returns a Promise

Correct.

```JavaScript
const promise = fetch("/users");
```

`fetch()` starts the HTTP request and immediately gives you a Promise representing the eventual result.

✅ 2. The server deals with HTTP request/response

Correct.

Conceptually:

```
Client                          Server
  |                               |
  | -------- HTTP Request ------->|
  |                               |
  |                           process request
  |                           query database
  |                               |
  |<------- HTTP Response --------|
  |                               |
```

The server might return:

```
200 OK
```

with data, or:

```
404 Not Found
```

with an error message.

Those are both **HTTP responses**.

`resolve()` and **`reject()`** aren't part of the HTTP protocol.

⚠️ 3. But "the server doesn't use Promises" isn't necessarily true

This part I'd change.

The **HTTP server doesn't need Promises as part of HTTP**, but the programming language/framework used to build the server absolutely **can use Promises**.

For example, a Node.js backend might do:

```JavaScript
const user = await database.findUser(id);
```

The database library could be returning a Promise.

So:

> **Promises are a JavaScript programming mechanism. HTTP request/response is a network protocol.**

They are separate concepts, although they often work together.

Now your most important question: `response`

You asked:

> Is **`response`** a keyword?

No.

This:

```JavaScript
fetch("/users")
    .then((response) => {
        console.log(response);
    });
```

could just as easily be:

```JavaScript
fetch("/users")
    .then((banana) => {
        console.log(banana);
    });
```

😂

Or:

```JavaScript
fetch("/users")
    .then((whatever) => {
        console.log(whatever);
```

```
});
```

They all work.

`response` is simply a **parameter name that the developer chose**.

So where does `response` come from?

This is the beautiful part.

Remember that **`fetch()`** returns a Promise.

Eventually, **`fetch()`** settles that Promise with a **Response object**.

Conceptually, imagine **`fetch()`** doing something like:

```
JavaScript
return new Promise((resolve, reject) => {

    // make HTTP request...

    if (networkWorked) {
        resolve(responseObject);
    } else {
        reject(error);
    }

});
```

So when you write:

```
JavaScript
.then((response) => {
    console.log(response);
})
```

you're basically saying:

> "When the Promise is fulfilled, take the value that it was fulfilled with and give that value to my function."

For example, if internally:

```
JavaScript
resolve(responseObject);
```

happens, then:

```
JavaScript
.then((response) => {
```

receives that **`responseObject`**.

So this:

```JavaScript
.then((response) => {
    console.log(response);
});
```

is basically:

```
Promise fulfilled
        ↓
   with this value
        ↓
 responseObject
        ↓
give it to my function
        ↓
(response) => { ... }
```

And **you chose to call that parameter `response`.**

The same thing applies to `error`

You asked:

> Where does **`error`** come from? Is it a keyword?

Again, **no**.

This:

```JavaScript
.catch((error) => {
    console.log(error);
});
```

could be:

```JavaScript
.catch((banana) => {
    console.log(banana);
});
```

The name doesn't matter.

What matters is:

```JavaScript
.catch((error) => ...)
```

means:

> "If this Promise gets rejected, give the rejection reason to this function."

For example, conceptually:

```JavaScript
reject(someError);
```

then:

```JavaScript
.catch((error) => {
    console.log(error);
});
```

receives **`someError`**.

And `data` is exactly the same thing

Consider:

```JavaScript
fetch("/users")
    .then((response) => {
        return response.json();
    })
    .then((data) => {
        console.log(data);
    });
```

You asked:

> Where did **`data`** come from?

Again, **you named it yourself**.

You could write:

```JavaScript
.then((banana) => {
    console.log(banana);
});
```

and it would work.

But there's another important thing happening here.

Look at this:

```JavaScript
.then((response) => {
    return response.json();
})
```

`response.json()` itself returns a **Promise**.

Eventually that Promise gets fulfilled with the actual parsed JSON.

So:

```
JavaScript
.then((data) => {
```

receives the value produced by the **previous `.then()`**.

That's why you can think of **`.then()`** as a chain.

This is the key mental model

Look at this:

```
JavaScript
fetch("/users")
    .then((response) => {
        return response.json();
    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
    });
```

Think of it like:

```
fetch()
  ↓
Promise
  ↓
fulfilled with Response object
  ↓
.then(response => ...)
  ↓
response.json()
  ↓
another Promise
  ↓
fulfilled with parsed JSON
  ↓
.then(data => ...)
  ↓
data = parsed JSON
```

The names **`response`** and **`data`** are just labels we gave to the values travelling through the chain.

Now your BIG question about `catch()`

You said:

> If the server sends 404, the Promise is fulfilled because we received a response. Then how does catch() catch that?

Excellent question.

And this is the most important correction to your current understanding:

`fetch()` does NOT reject its Promise just because the server returns 404 or 500.

For example:

```
Server
   ↓
HTTP 404 Not Found
```

`fetch()` normally says:

> "I successfully communicated with the server and received an HTTP response."

Therefore:

```JavaScript
fetch("/something")
    .then((response) => {
        console.log(response.status);
    })
    .catch((error) => {
        console.log(error);
    });
```

will normally execute **`.then()`** and print:

```
404
```

The `.catch()` does not automatically run.

Then how do we treat 404 as an error?

We explicitly check it.

The **`Response`** object has a property called:

```JavaScript
response.ok
```

For successful HTTP statuses such as 200:

```JavaScript
response.ok === true
```

For 404:

```JavaScript
response.ok === false
```

So we can write:

```JavaScript
fetch("/users/999")
    .then((response) => {

        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }

        return response.json();
    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
    });
```

Now something interesting happens.

Suppose server says:

```
404 Not Found
```

Then:

```JavaScript
response.ok
```

is **`false`**.

So we do:

```JavaScript
throw new Error(...)
```

And **throwing inside `.then()` causes the Promise returned by that `.then()` to become rejected.**

Therefore:

```
Server
  ↓
404 response
  ↓
fetch Promise fulfilled
  ↓
.then(response)
  ↓
response.ok === false
  ↓
throw Error
  ↓
Promise becomes rejected
  ↓
.catch(error)
  ↓
console.log(error)
```

That's how you get the error into `.catch()`.

So your final mental model should be this

There are **three different concepts** here:

1. Network failure

Something prevents `fetch()` from successfully obtaining an HTTP response.

For example, in simplified terms:

```
Internet/network problem
        ↓
fetch Promise rejects
        ↓
.catch()
```

2. HTTP error response

The server successfully responds:

```
404 Not Found
```

or:

```
500 Internal Server Error
```

The HTTP request worked, so:

```
fetch Promise fulfills
        ↓
.then(response)
```

You inspect:

```JavaScript
response.ok
response.status
```

and decide what to do.

---

3. Your own application error handling

You can turn that HTTP error into a rejected Promise yourself:

```JavaScript
if (!response.ok) {
    throw new Error("Something went wrong");
```

```
}
```

Then:

```
throw
 ↓
rejected Promise
 ↓
.catch()
```

And one last thing: `.then()` parameters

This is worth memorizing because it will appear **everywhere** in JavaScript:

```JavaScript
promise.then((value) => {
    // use value
});
```

`value` is **not a keyword**.

It's simply the value with which the previous Promise was fulfilled.

Similarly:

```JavaScript
promise.catch((error) => {
    // use error
});
```

`error` is **not a keyword**.

It's simply the reason/value with which the previous Promise was rejected.

And:

```JavaScript
promise
    .then((response) => {
        return response.json();
    })
    .then((data) => {
        console.log(data);
    });
```

means:

> **Promise #1 fulfilled with a Response → put that value into `response`. Then `response.json()` gives us Promise #2 → when that fulfills, put its value into `data`.**

Once you understand **"the parameter is just receiving the value passed from the previous Promise"**, **`.then()`** and **`.catch()`** become much less mysterious.

6. Now there are few questions of mine. Like you said that when the server sends 404, the promise is fulfilled, that is considered a valid response, and our fetch keyword receives it, and then returns a promise. Then we need to handle it with .then and .catch.
And you are saying that the response object has a property called .ok. So, ok is true for status code like 200 and false for 400. But, you know, like there are series of status codes like 100 and so on, 200 so on, 300 so on, 400 so on, 500 so on. So like, who is defining all of these, like, whether 200 is okay while 400 is not?

And then you told me that now based on the response that you received from fetch, then you can apply .then to convert that response according to our need. For example, if the response is not okay, then you can throw a new error and print the response status as the error, like in the error itself, and then return response.json. Basically, whatever you have created, this new, then return it. And then this error will be catched by .catch. But there is one small point that I didn't understand quite well that the return you are saying that response.json, is it of the original response that we received in the fetch or the one that we made up?

And you didn't even tell me like properly whether I was right or wrong about this one intuition that if there is any network failure in between the server and the client, then that is also successfully cached by .catch. And this is the one main reason for .catch, and another use that we can do it manually by, you know, taking the server response and then raising error manually in the code for .catch to catch it.

One more question that is in .then and .catch, I can see that you are passing an arrow function in the parentheses that you were saying response data and error there, but just the variable names for the parameter. Like, they are not keywords. You told that and I understood that point. But the one point is, while when we have a, you know, parameter, you know, function, in any function, including arrow function, we need to pass arguments to those parameters. But here, like in .then and .catch, we are not passing arguments in those, you know, in those arrow functions inside .then and .catch. Then how they are receiving those arguments correctly? Like from fetch, the response came, but we didn't pass that response to .catch response. So how it get that?
Show moreShow less

Yes — these are exactly the right questions. You're now getting into the **mechanics behind callbacks and Promise chaining**, which is the part that makes `.then()`/`.catch()` really click.

Let me answer each one separately.

---

1. Who decides that 200 is "okay" and 400 is "not okay"?

This comes from the **HTTP standard**, not from JavaScript and not from `fetch()`.

HTTP defines status-code ranges:

Range	Meaning
100–199	Informational
200–299	Successful
300–399	Redirection
400–499	Client errors
500–599	Server errors

For example:

```
200 OK
201 Created
204 No Content

301 Moved Permanently
302 Found

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
```

These meanings are defined by the **HTTP specification**.

So when a server sends:

```
HTTP/1.1 404 Not Found
```

the browser knows what **`404`** means because HTTP defines it.

And **`fetch()`** follows the web platform's rules for interpreting that response.

One subtle point

`response.ok` isn't literally:

> "Is this response good according to my application?"

It's essentially:

> "Is the HTTP status in the successful range (**`200–299`**)?"

So:

```javascript
response.ok
```

is **`true`** for **`200–299`**, and **`false`** otherwise.

That means even a `302` redirect isn't `ok === true`, although `fetch()` may automatically follow redirects depending on how it's configured.

---

2. Your question about `response.json()`

This is an **excellent question**, because there are actually two different "responses" that can get confusing.

Suppose we have:

```JavaScript
fetch("/users")
    .then((response) => {

        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }

        return response.json();

    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
    });
```

You're asking:

> Is `response.json()` using the original response from `fetch()`, or the error/response we created ourselves?

It's using the **original `Response` object received from `fetch()`**.

Let's follow it.

Step 1

```JavaScript
fetch("/users")
```

gives you a Promise.

Eventually, suppose the server sends:

```
200 OK
Content-Type: application/json

{
    "name": "John",
    "age": 25
}
```

The Promise from `fetch()` is fulfilled with a **Response object**.

So:

```JavaScript
.then((response) => {
```

receives that Response object.

Step 2

Inside that function:

```JavaScript
if (!response.ok) {
    throw new Error(...);
}
```

Suppose status is **`200`**.

Therefore:

```JavaScript
response.ok === true
```

so we don't throw anything.

Then:

```JavaScript
return response.json();
```

Here **`response`** is **still the original Response object**.

`.json()` reads the response body and parses the JSON.

Importantly:

`response.json()` itself returns a Promise.

So you now have:

```
fetch()
  ↓
Promise
  ↓
Response object
  ↓
response.json()
  ↓
another Promise
  ↓
parsed JavaScript object
```

That's why the next **`.then()`** gets **`data`**.

---

3. What happens if the response is 404?

Suppose:

```
404 Not Found
```

Then:

```
JavaScript
response.ok === false
```

So:

```
JavaScript
throw new Error(`HTTP error: ${response.status}`);
```

runs.

At that point, **we never reach**:

```
JavaScript
return response.json();
```

because **`throw`** immediately exits that callback.

The chain becomes rejected:

```
fetch()
  ↓
fulfilled with Response
  ↓
.then(response)
  ↓
response.ok === false
  ↓
throw new Error(...)
  ↓
Promise becomes rejected
  ↓
.catch(error)
```

So **`.catch()`** receives the **`Error`** that **we created**.

That's different from **`response.json()`**.

---

4. Now your question about network failure

You were asking whether your intuition was correct.

Yes. ✅

This is one of the primary purposes of **`.catch()`** when using **`fetch()`**.

For example, imagine:

```JavaScript
fetch("/users")
    .then((response) => {
        return response.json();
    })
    .catch((error) => {
        console.log(error);
    });
```

If the browser can't successfully complete the fetch operation — for example, due to certain network-level failures — the Promise returned by **`fetch()`** can be **rejected**.

Then:

```JavaScript
.catch(...)
```

handles that rejection.

So you can think:

```
NETWORK FAILURE
      ↓
fetch Promise rejects
      ↓
.catch()
```

That's one situation.

But `.catch()` isn't only for network failures.

This is extremely important.

Suppose you write:

```JavaScript
fetch("/users")
    .then((response) => {

        if (!response.ok) {
            throw new Error("HTTP error");
        }

        return response.json();

    })
    .catch((error) => {
        console.log(error);
    });
```

Now **`.catch()`** can receive:

1. A rejection from **`fetch()`**

2. An error you explicitly `throw`
3. An error thrown inside an earlier `.then()`
4. A rejection from another Promise in the chain

So `.catch()` is more generally:

> **"Handle a rejected Promise/error that occurs anywhere upstream in this chain."**

---

5. Now your MOST interesting question: "How does `.then()` get the argument?"

This is the part I really want you to understand.

You said:

> Normally, if a function has a parameter, we need to pass an argument. But here we're not passing anything.

You're absolutely right.

Consider a normal function:

```JavaScript
function greet(name) {
    console.log(name);
}

greet("Rahul");
```

We explicitly pass:

```
"Rahul"
   ↓
name
```

So you're wondering why this:

```JavaScript
promise.then((response) => {
    console.log(response);
});
```

doesn't require:

```JavaScript
.then(something)
```

Great question.

The answer is:

`.then()` itself calls your function later and passes the argument for you.

That's the whole point of a **callback function**.

Let's forget Promises for a moment

Imagine you have:

```JavaScript
function doSomething(callback) {

    let result = "Hello";

    callback(result);
}
```

Now you call:

```JavaScript
doSomething((value) => {
    console.log(value);
});
```

You did **not** manually do:

```JavaScript
(value)("Hello")
```

Instead, **`doSomething()`** itself does:

```JavaScript
callback(result);
```

So the function you supplied:

```JavaScript
(value) => {
    console.log(value);
}
```

receives **`"Hello"`**.

The flow is:

```
doSomething()
      ↓
creates result = "Hello"
      ↓
callback(result)
      ↓
(value) receives "Hello"
```

`.then()` works on the same principle

You can imagine a simplified Promise implementation doing something conceptually like:

```JavaScript
promise.then = function(callback) {

    // wait until Promise is fulfilled

    callback(resolvedValue);

};
```

So when you write:

```JavaScript
promise.then((response) => {
    console.log(response);
});
```

you're basically saying:

> "Hey Promise, when you're fulfilled, call this function and give it the value you were fulfilled with."

And internally, conceptually:

```JavaScript
callback(responseValue);
```

happens.

Therefore:

```JavaScript
(response) => {
```

receives that value.

And `.catch()` works the same way

Conceptually:

```JavaScript
promise.catch = function(callback) {

    // wait until Promise is rejected

    callback(rejectionReason);

};
```

So:

```JavaScript
.catch((error) => {
    console.log(error);
});
```

means:

> "When you get rejected, call this function and give me the rejection reason."

The Promise machinery effectively does:

```JavaScript
callback(error);
```

for you.

This is the connection I want you to remember

You were previously asking:

> "Where does **`response`** come from?"

Now you can see the complete answer:

```JavaScript
fetch("/users")
    .then((response) => {
        console.log(response);
    });
```

Conceptually:

```
fetch()
  ↓
Promise
  ↓
Promise eventually gets fulfilled
  ↓
Promise has a value
  ↓
.then() calls YOUR callback
  ↓
callback(responseValue)
  ↓
(response) receives responseValue
```

You don't call the callback. The Promise system calls it.

That's why it's called a **callback**.

And this explains the whole chain

Take this:

```JavaScript
fetch("/users")
    .then((response) => {
        return response.json();
    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
    });
```

Think of it as a series of callbacks being called automatically:

```
fetch()
  │
  │ eventually fulfills
  ↓
.then( callback1 )
  │
  │ callback1 gets Response
  ↓
response.json()
  │
  │ returns another Promise
  ↓
.then( callback2 )
  │
  │ callback2 gets parsed JSON
  ↓
data
```

And if something rejects:

```
                          ┌── network failure
                          │
fetch() ──────────────────┤
                          │
                          ├── throw inside .then()
                          │
                          └── response.json() rejects
                                    ↓
                                .catch()
                                    ↓
                          catch callback gets
                               the error
```

The one sentence I'd like you to remember

`.then()` and `.catch()` don't need you to manually pass the arguments because they are callback-registration methods: you give them a function, and the Promise machinery calls that function later with the appropriate value.

So when you write:

```JavaScript
.then((response) => ...)
```

you're **not calling** that arrow function.

You're **giving the function to `.then()` and saying, "Call this later when the Promise fulfills, and give it the fulfilled value."**

And:

```JavaScript
.catch((error) => ...)
```

means:

> "Call this later if the Promise becomes rejected, and give it the rejection reason."

That callback concept is actually **bigger than Promises**. Once you understand callbacks properly, **`.then()`**, **`.catch()`**, event listeners, timers, array methods like **`.map()`**, and a lot of JavaScript's asynchronous behavior become much easier to understand.

7. Okay, now I have understood almost all the things. Just two things I want to know more. That is, first, you said that, you know, the status code and all that comes, that's not a part of JavaScript, but the HTTP header itself. And it is defined that whether to consider this range of status code as request.ok true or false. Okay? So, and I think there might be some stored values for every range from 100 to 199, 200 to 299, till 500 and so on. So, like, my question is, does that mean like response.ok, so basically it is happening something like this. You tell me whether I am right or wrong. So basically, response is an object, first of all. And it has a property named ok. And ok stores a value like we have an object key-value pair. So ok here is the key which stores the value true or false. And if the response, like we are just checking that, is this the current value is true or false, and making use of it in .then later on to throw new errors if the response is not okay. My second question is, in entire all of the things we have learned so far, there is not even a single use of reject and resolve so far. Like, so what the hell is the use of those two things? Like, I haven't seen any of those use. I can see that .then, .catch, and they works on, you know, the fetch, whatever the fetch receives as a response, .then and .catch works on them. .catch is very optimized that it can catch any error, even the error produced from the chaining of .then in between, or if not, then directly from the response the fetch received, or even if the fetch fails, then that also is cached by .catch. And if we are just talking about response, the error inside the response, I have an intuition that it might not even, you know, if we try to work around, we can even handle that error with .then as well, because it's a fulfilled response and we can handle it. Just my intuition, just you tell me whether it is possible or not. Obviously, I am going to follow convention. But I haven't seen, like, resolve and reject thing being used here at all.
Show moreShow less

Yes — your understanding of **`response.ok`** is **correct**, with one small refinement. And your second question gets to the heart of **why Promises exist in the first place**.

Let's take them one at a time.

1. Is `response.ok` just a property containing `true` or `false`?

Yes. Exactly. ✅

When **`fetch()`** fulfills its Promise, it gives you a **`Response`** object.

You can conceptually think of that object as something like:

```JavaScript
{
    ok: true,
    status: 200,
    statusText: "OK",
    headers: ...,
    body: ...
}
```

That's simplified — the real **`Response`** object has more behavior and properties — but your mental model is correct.

So:

```JavaScript
response.ok
```

means:

> "Give me the value stored in the **`ok`** property."

And that value is a boolean:

```JavaScript
true
```

or:

```JavaScript
false
```

For **`fetch()`**, **`ok`** is **`true`** when the HTTP status is in the **200–299 range**.

So:

```JavaScript
if (response.ok) {
    // success
}
```

is basically checking:

```JavaScript
```

```
if (true) {
    // ...
}
```

And:

```
JavaScript
if (!response.ok) {
    // error
}
```

means:

```
JavaScript
if (response.ok === false) {
    // ...
}
```

One small correction to your wording

You said:

> "HTTP defines whether to consider this range as request.ok true or false."

The HTTP standard defines what the **status codes mean**. The `ok` property is a **web-platform/Fetch API convenience** built on top of those HTTP semantics.

So:

```
HTTP standard
    ↓
200 = successful response
404 = not found
500 = server error
    ↓
Fetch API
    ↓
Response.ok
    ↓
true for 200–299
false otherwise
```

Your object/key-value mental model is absolutely the right way to think about `response.ok`.

---

2. Now: "Where the hell are `resolve` and `reject`?" 😂

This is actually the most important realization.

You've been using Promises **without directly creating them**.

That's because:

> **`fetch()` creates the Promise for you.**

Remember your earlier learning example:

```JavaScript
let promise = new Promise((resolve, reject) => {

    if (somethingWorked) {
        resolve(value);
    } else {
        reject(error);
    }

});
```

You were the person **creating the Promise**.

Therefore, you had access to:

```JavaScript
resolve()
reject()
```

But when you use:

```JavaScript
fetch(url)
```

you aren't creating the Promise.

The implementation of **`fetch()`** does that work for you.

Conceptually, you can imagine something like:

```JavaScript
function fetch(url) {

    return new Promise((resolve, reject) => {

        // perform network operation...

        if (networkOperationSucceeded) {
            resolve(response);
        } else {
            reject(networkError);
        }

    });

}
```

This is **not the actual implementation of `fetch()`** — it's a simplified mental model.

But it explains exactly why you don't see **`resolve()`** and **`reject()`**.

Think about it from the two sides

When you were learning how to **create** a Promise:

```JavaScript
```

```
new Promise((resolve, reject) => {
    // ...
});
```

you were on the **producer side**.

You were saying:

> "I'm going to perform some asynchronous operation, and I'll eventually tell the Promise whether it succeeded or failed."

So you get:

```
            YOU
             |
    ┌────────┴────────┐
    ↓                 ↓
 resolve           reject
    ↓                 ↓
fulfilled         rejected
```

When you're using **`fetch()`**:

```
JavaScript
fetch(url)
```

you're on the **consumer side**.

Somebody else — the implementation of **`fetch()`** — is producing/settling the Promise.

You just consume it:

```
JavaScript
fetch(url)
    .then(...)
    .catch(...);
```

So:

```
              fetch()
                |
    ┌───────────┴──────────┐
    ↓                      ↓
 resolve                reject
    ↓                      ↓
 .then()               .catch()
```

You don't need to see the **`resolve()`** and **`reject()`** because **they're inside the machinery you're using**.

And this is why your teacher said you normally don't create Promises

Imagine you had to manually do this every time you wanted an API request:

```JavaScript
new Promise((resolve, reject) => {

    // open network connection
    // send HTTP request
    // wait for response
    // handle connection errors
    // process response
    // resolve/reject appropriately

});
```

That would be horrible.

Instead:

```JavaScript
fetch(url)
```

does all of that for you.

Then you simply say:

```JavaScript
.then(...)
.catch(...)
```

And YES — your intuition about 404 is correct

You said something very important:

> "If we're talking about an error inside the response, I think we could handle that with **`.then()`** because it's a fulfilled response."

Yes. Absolutely. ✅

In fact, that's exactly what normally happens with **`fetch()`**.

Suppose the server returns:

```
404 Not Found
```

`fetch()` gives you the Response object.

So:

```JavaScript
fetch(url)
    .then((response) => {
        console.log(response.status);
    });
```

will execute.

You can then do:

```JavaScript
fetch(url)
    .then((response) => {

        if (response.status === 404) {
            console.log("User wasn't found");
        }

    });
```

No `.catch()` is required for the `404` itself.

Why?

Because:

```
404
 ↓
valid HTTP response
 ↓
fetch Promise fulfilled
 ↓
.then()
```

You can even handle ALL HTTP statuses inside `.then()`

For example:

```JavaScript
fetch(url)
    .then((response) => {

        if (response.ok) {
            console.log("Success!");
        } else {
            console.log("HTTP error:", response.status);
        }

    });
```

That works.

But there's a reason people commonly do:

```JavaScript
fetch(url)
    .then((response) => {

        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }

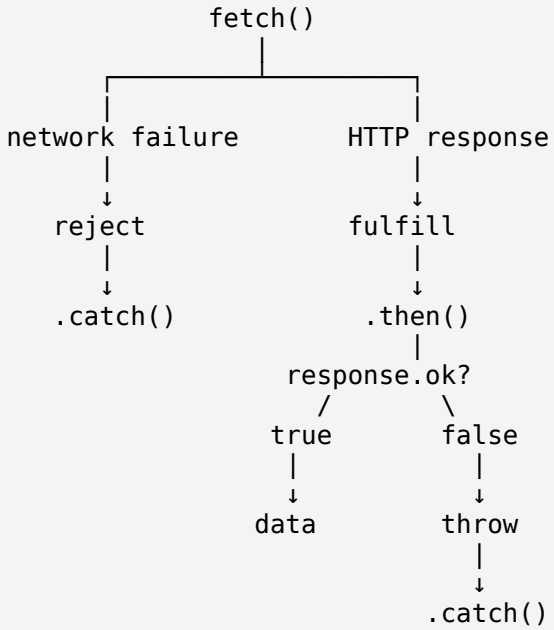
        return response.json();

    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
```

```
});
```

It gives you a **single error-handling path**.

You can think of it like this:



So **`.catch()`** becomes your common place for handling things that you have decided should be treated as errors.

So what is the actual purpose of `resolve` and `reject`?

This is the key takeaway:

`resolve()` and `reject()` are for the producer of a Promise.

`.then()` and `.catch()` are for the consumer of a Promise.

That's the distinction you've been missing.

For example, suppose **you create your own function**:

```
JavaScript
function getUser() {

    return new Promise((resolve, reject) => {

        // pretend we're getting a user

        const user = { name: "John" };

        if (user) {
            resolve(user);
        } else {
            reject("User not found");
        }

    });
```

```
}
```

Now someone else uses your function:

```JavaScript
getUser()
    .then((user) => {
        console.log(user);
    })
    .catch((error) => {
        console.log(error);
    });
```

The person **who created the Promise** uses:

```JavaScript
resolve()
reject()
```

The person **using the Promise** uses:

```JavaScript
.then()
.catch()
```

And in a real application, the two people could even be different pieces of code, libraries, or teams.

The complete picture

You've now basically learned this:

```
                PROMISE
                   |
       ┌───────────┴──────────┐
       |                      |
   PRODUCER                CONSUMER
       |                      |
       |                      |
  resolve(value)          .then(value => ...)
  reject(error)           .catch(error => ...)
       |                      |
       ↓                      ↓
  settle Promise          handle result
```

For **`fetch()`**:

```
            fetch implementation
                   |
            creates Promise
                   |
          ┌────────┴─────────┐
          ↓                  ↓
      resolve             reject
    (Response)          (network error)
       |                      |
```

```
        ↓                    ↓
    .then()              .catch()
```

And the **HTTP 404 case** sits slightly differently:

```
Server
  ↓
404 HTTP response
  ↓
fetch receives valid HTTP response
  ↓
Promise fulfills
  ↓
.then(response)
  ↓
response.ok === false
  ↓
you decide what to do
```

So honestly, your intuition at this point is **very good**.

The one big distinction to permanently remember is:

> **HTTP errors and Promise rejections are not the same thing.** A `404` is an HTTP-level error, but `fetch()` normally fulfills its Promise because it successfully received an HTTP response. A network failure, on the other hand, can cause `fetch()` to reject. You can also take a `404` yourself and `throw` an error, which turns it into a rejected Promise and sends it to `.catch()`.**

And `resolve/reject` **haven't disappeared** — you've simply moved from the side that **creates/settles** Promises to the side that **consumes** them.

8. Okay, now just one last request, like one more point I need to be clarified in my mind. That is the production side of promises. Like, now I'm not talking about handling the API response, but how APIs themselves are created. I think promises are used over there. So now my question is, how resolve and reject are used over there? Like, you said that in the HTTP request, there are many metadata that comes as a response from the server. So like all those responses are bundled inside the reject and resolve, and then passed as a response here, or what? Like, and if possible, give me one example of such code.
Show moreShow less

Yes — and this is a **very good final piece to understand**. The main thing to separate is:

> **Promises are not what an API uses to communicate over HTTP. Promises are something the server-side JavaScript code can use internally while it is doing asynchronous work.**

And `resolve()`/`reject()` don't bundle HTTP metadata themselves. **You decide what value to pass to them.**

Let's make this concrete.

Imagine you're creating your own API

Suppose we have a Node.js server with an endpoint:

```
GET /users/123
```

A simplified server might look like:

```JavaScript
app.get("/users/:id", async (req, res) => {

    try {
        const user = await getUserFromDatabase(req.params.id);

        if (!user) {
            return res.status(404).json({
                message: "User not found"
            });
        }

        res.status(200).json({
            message: "Success",
            data: user
        });

    } catch (error) {

        res.status(500).json({
            message: "Something went wrong"
        });

    }

});
```

Notice something important:

There are **two completely different things** happening here.

Promise side

```JavaScript
const user = await getUserFromDatabase(...);
```

`getUserFromDatabase()` might return a Promise.

HTTP side

```JavaScript
res.status(200).json(...)
```

This sends the HTTP response to the client.

So the Promise isn't being sent over the network.

Let's go one level deeper

Imagine we create **`getUserFromDatabase()`** ourselves:

```
JavaScript
function getUserFromDatabase(id) {

    return new Promise((resolve, reject) => {

        database.query(
            "SELECT * FROM users WHERE id = ?",
            [id],
            (error, result) => {

                if (error) {
                    reject(error);
                } else {
                    resolve(result);
                }

            }
        );

    });
}
```

Now you can finally see the **production-side use of `resolve()` and `reject()`**.

The database operation is asynchronous.

So we create a Promise.

If the database operation fails:

```
JavaScript
reject(error);
```

If it succeeds:

```
JavaScript
resolve(result);
```

The Promise is carrying the **result of the database operation inside the server's JavaScript memory**.

It is **not being transmitted to the frontend**.

Then the API converts that result into HTTP

Our server code receives the result:

```
JavaScript
const user = await getUserFromDatabase(id);
```

If the Promise was fulfilled:

```
database
   ↓
result
```

```
   ↓
resolve(result)
   ↓
Promise fulfilled
   ↓
await
   ↓
user
```

Then the server decides what HTTP response to send:

```JavaScript
res.status(200).json({
    data: user
});
```

Now **that** is what travels across the network.

And if the database fails?

Suppose:

```JavaScript
database.query(...)
```

fails.

Then:

```JavaScript
reject(error);
```

happens.

The Promise becomes rejected.

Our **`try/catch`** catches it:

```JavaScript
try {

    const user = await getUserFromDatabase(id);

} catch (error) {

    res.status(500).json({
        message: "Something went wrong"
    });

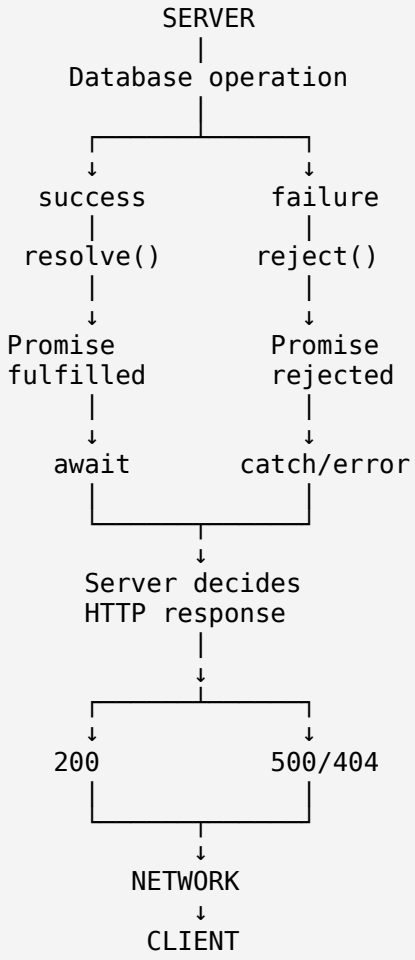
}
```

And **now** the server sends an HTTP response:

```
HTTP/1.1 500 Internal Server Error
```

```
{
    "message": "Something went wrong"
}
```

So the flow is:



Now your question about "bundling the metadata"

This is the important correction:

> **No, `resolve()` doesn't automatically bundle the HTTP status code, headers, body, etc.**

`resolve()` simply takes **whatever value you give it**.

For example:

```
JavaScript
resolve(result);
```

Then the Promise's fulfilled value is **`result`**.

You could instead do:

```
JavaScript
resolve({
    data: result,
    statusCode: 200
```

```
});
```

Now the Promise's value is that object.

Or:

```JavaScript
resolve({
    user: result,
    message: "User found"
});
```

Again, completely up to you.

The Promise doesn't know what an HTTP response is.

Here's a very simplified API from beginning to end

Imagine:

Client

```JavaScript
fetch("/users/123")
    .then(response => response.json())
    .then(data => {
        console.log(data);
    });
```

↓

Server

```JavaScript
app.get("/users/:id", async (req, res) => {

    try {

        const user = await getUserFromDatabase(req.params.id);

        res.status(200).json({
            data: user
        });

    } catch (error) {

        res.status(500).json({
            message: "Database failed"
        });

    }

});
```

↓

Database function

```
JavaScript
function getUserFromDatabase(id) {

    return new Promise((resolve, reject) => {

        database.query(
            "SELECT * FROM users WHERE id = ?",
            [id],
            (error, result) => {

                if (error) {
                    reject(error);
                } else {
                    resolve(result);
                }

            }
        );

    });
}
```

Now you can see **exactly where `resolve()` and `reject()` belong**.

They're used to communicate between **asynchronous pieces of JavaScript code inside the server**.

They aren't sent to the client.

One final mental model

Think of **`resolve()`** and **`reject()`** as **internal delivery mechanisms**:

```
SERVER'S INTERNAL WORLD

Database ——success——> resolve(result) ——> await
Database ——failure——> reject(error)  ——> catch
```

Then the server takes that result and creates an **HTTP response**:

```
SERVER
  ↓
res.status(200).json(data)
  ↓
HTTP
  ↓
INTERNET
  ↓
CLIENT
```

So there are **two completely separate communication systems**:

Inside JavaScript:

```
Promise → resolve/reject → then/catch/await
```

Between client and server:

```
HTTP request → HTTP response → status/headers/body
```

They often work together, but **a Promise itself never travels across the network**. That's probably the single most important thing to lock into your mental model now.

9. Okay, so now just do one thing. All the doubts that I had, all the points that you explained me so far regarding promises and all, everything, summarize it. All of them. Put all of the points in brief so that all of them can fit in - and summarize all of the thing that we have studied above in detail don't miss any single point
Show moreShow less

Absolutely. Here's a **complete consolidated set of notes** covering essentially everything we discussed — from what a Promise is, to `fetch()`, HTTP status codes, `.then()`, `.catch()`, callbacks, `resolve/reject`, API servers, and the production side.

JavaScript Promises — Complete Summary

1. What is a Promise?

A **Promise is a JavaScript object** that represents the eventual result of an asynchronous operation.

It can be in one of three states:

```
Pending     → operation is still going
Fulfilled   → operation succeeded
Rejected    → operation failed
```

You can manually create one with:

```JavaScript
const promise = new Promise((resolve, reject) => {
    // some work
});
```

`Promise` is a built-in JavaScript constructor, so `new Promise(...)` creates a Promise object.

---

2. What are `resolve` and `reject`?

When you create a Promise:

```JavaScript
new Promise((resolve, reject) => {
    // ...
});
```

JavaScript provides two functions to your executor function:

```
resolve()
reject()
```

They are **not keywords**. They are functions provided to the executor.

You use them to settle the Promise:

```JavaScript
resolve(value);
```

means:

> "The operation succeeded. Fulfill this Promise with this value."

And:

```JavaScript
reject(error);
```

means:

> "The operation failed. Reject this Promise with this reason/error."

---

3. The Promise doesn't automatically know whether something succeeded

This was one of your original doubts.

The Promise doesn't look at your code and magically decide:

> "Oh, this was successful, so I'll call resolve."

You or the library performing the operation decides.

For example:

```JavaScript
const promise = new Promise((resolve, reject) => {

    const age = 20;

    if (age >= 18) {
        resolve("You are allowed");
    } else {
        reject("You are not allowed");
    }

});
```

Here **your conditional determines** whether `resolve()` or `reject()` gets called.

4. Your simple learning examples

This:

```JavaScript
const promise = new Promise((resolve, reject) => {
    console.log("I am a promise");

    resolve("success");
});
```

means:

```
Create Promise
      ↓
Execute executor immediately
      ↓
console.log(...)
      ↓
resolve("success")
      ↓
Promise becomes fulfilled
```

Similarly:

```JavaScript
const promise2 = new Promise((resolve, reject) => {
    console.log("I am a promise - 2");

    reject("failed");
});
```

means:

```
Create Promise
      ↓
Execute executor immediately
      ↓
reject("failed")
      ↓
Promise becomes rejected
```

There is no automatic "success detection" happening.

---

5. Promise executors run immediately

This:

```JavaScript
new Promise((resolve, reject) => {
    console.log("Hello");
});
```

prints **`"Hello"`** immediately when the Promise is created.

A Promise itself does **not** mean:

> "This code will automatically run later."

Promises are generally used to represent the result of asynchronous operations, but the executor function itself begins executing immediately.

---

6. `resolve()` and `reject()` accept a value

For example:

```JavaScript
resolve("success");
```

The Promise is fulfilled with **`"success"`**.

You can also resolve with an object:

```JavaScript
resolve({
    message: "Success",
    statusCode: 201
});
```

Similarly:

```JavaScript
reject({
    message: "Failed",
    statusCode: 404
});
```

The Promise is rejected with that object.

Important:

`resolve()` and **`reject()`** effectively use **one value**.

So:

```JavaScript
resolve("success", 123);
```

doesn't give two separate values.

The second argument is ignored.

If you need multiple pieces of information:

```JavaScript
resolve({
    message: "success",
    number: 123
});
```

7. A rejected Promise should be handled

If you do:

```JavaScript
const promise = new Promise((resolve, reject) => {
    reject({
        message: "Failed",
        statusCode: 404
    });
});
```

but never attach a rejection handler:

```JavaScript
promise.catch(...);
```

the browser can report something like:

```
Uncaught (in promise) #<Object>
```

because the Promise was rejected and nobody handled that rejection.

---

8. `.then()` and `.catch()` are used to consume a Promise

If a Promise succeeds:

```JavaScript
promise.then((result) => {
    console.log(result);
});
```

If it rejects:

```JavaScript
promise.catch((error) => {
    console.log(error);
});
```

So:

```
resolve(value)
      ↓
fulfilled Promise
      ↓
```

```
.then(...)
```

and:

```
reject(error)
     ↓
rejected Promise
     ↓
.catch(...)
```

9. `result`, `response`, `data`, and `error` are NOT keywords

This was another major doubt.

In:

```JavaScript
promise.then((result) => {
    console.log(result);
});
```

`result` is simply a **parameter name** chosen by the developer.

You could write:

```JavaScript
promise.then((banana) => {
    console.log(banana);
});
```

It works exactly the same.

Likewise:

```JavaScript
.catch((error) => {
    console.log(error);
});
```

`error` isn't a special JavaScript keyword.

You could call it:

```JavaScript
.catch((somethingWentWrong) => {
    console.log(somethingWentWrong);
});
```

Same thing.

---

10. But where do those arguments come from?

This was one of your most important questions.

You correctly noticed:

> "Normally, if a function has a parameter, somebody needs to pass an argument."

Exactly.

The difference here is that **you aren't calling the callback yourself**.

You give the function to **`.then()`**:

```
JavaScript
.then((response) => {
    console.log(response);
});
```

And the Promise machinery calls that function later.

Conceptually, think:

```
JavaScript
callback(resolvedValue);
```

So:

```
JavaScript
.then((response) => {
```

means roughly:

> "When the Promise fulfills, call this function and pass the fulfilled value to it."

Similarly:

```
JavaScript
.catch((error) => {
```

means:

> "When the Promise rejects, call this function and pass the rejection reason to it."

This is the concept of a **callback**.

---

11. `fetch()` returns a Promise

When you write:

```
JavaScript
const promise = fetch(url);
```

`fetch()` starts an HTTP request and returns a Promise.

So:

```
JavaScript
fetch(url)
    .then(...)
    .catch(...);
```

means you're **consuming the Promise returned by `fetch()`**.

You usually don't see `resolve()` and `reject()` because **you didn't create this Promise**.

The implementation of `fetch()` handles that internally.

---

12. Why you don't normally write `resolve()` and `reject()` when using `fetch()`

When you manually create a Promise:

```
JavaScript
new Promise((resolve, reject) => {
    // ...
});
```

you're the **producer** of the Promise.

You decide:

```
JavaScript
resolve(...)
```

or:

```
JavaScript
reject(...)
```

But with:

```
JavaScript
fetch(url)
```

the `fetch()` implementation is the **producer**.

Conceptually, you can imagine:

```
JavaScript
function fetch(url) {

    return new Promise((resolve, reject) => {
```

```
        // perform network operation

        if (networkSucceeded) {
            resolve(response);
        } else {
            reject(error);
        }

    });

}
```

This is only a simplified mental model, **not the actual implementation**.

You are the **consumer**:

```
JavaScript
fetch(url)
    .then(...)
    .catch(...);
```

So:

```
Producer:
resolve() / reject()

Consumer:
.then() / .catch() / await
```

13. Promises and HTTP are two different concepts

This distinction is extremely important.

Promise

A JavaScript mechanism:

```
Promise
 ↓
resolve/reject
 ↓
then/catch/await
```

HTTP

A network protocol:

```
HTTP Request
 ↓
Server
 ↓
HTTP Response
```

HTTP defines things such as:

- status codes

- headers
- request methods
- response body
- etc.

A Promise does **not travel across the network**.

---

14. What happens when `fetch()` makes an API request?

Conceptually:

```
Your JavaScript
      ↓
    fetch()
      ↓
HTTP Request
      ↓
Network
      ↓
API Server
      ↓
Server processes request
      ↓
HTTP Response
      ↓
Network
      ↓
Browser
      ↓
fetch() settles its Promise
```

15. What does the server do?

A server receives an HTTP request.

Conceptually:

```
Request
  ↓
Server
  ↓
Authentication / validation
  ↓
Business logic
  ↓
Database query
  ↓
Create HTTP response
```

For example:

```
GET /users/123
```

The server might look for user **123**.

If found:

```
200 OK
{
    "name": "John"
}
```

If not found:

```
404 Not Found
{
    "message": "User not found"
}
```

Both are **HTTP responses**.

---

16. The server does not have to use Promises for HTTP

HTTP itself does not require Promises.

However, if the backend is written in JavaScript/Node.js, it may absolutely use Promises internally.

For example:

```
JavaScript
const user = await database.findUser(id);
```

The database library may return a Promise.

So:

> **The server can use Promises internally, but the HTTP response itself is not a Promise.**

---

17. HTTP status codes

HTTP defines standardized status-code categories:

```
100–199 → Informational
200–299 → Successful
300–399 → Redirection
400–499 → Client errors
500–599 → Server errors
```

Examples:

```
200 OK
201 Created
204 No Content

301 Moved Permanently
302 Found

400 Bad Request
```

```
401 Unauthorized
403 Forbidden
404 Not Found

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
```

These meanings come from the **HTTP specifications**, not JavaScript.

---

18. What is `response` in `fetch()`?

Consider:

```JavaScript
fetch(url)
    .then((response) => {
        console.log(response);
    });
```

`response` is simply a parameter name.

The Promise returned by **`fetch()`** is eventually fulfilled with a **Response object**.

Conceptually:

```
fetch()
  ↓
Promise fulfilled
  ↓
Response object
  ↓
.then(callback)
  ↓
callback(responseObject)
  ↓
response parameter receives it
```

So:

```JavaScript
.then((response) => ...)
```

doesn't mean **`response`** is a keyword.

It's simply:

```JavaScript
.then((someValue) => ...)
```

where the Promise machinery supplies **`someValue`**.

---

19. The `Response` object has properties

A simplified mental model is:

```JavaScript
{
    ok: true,
    status: 200,
    statusText: "OK",
    headers: ...,
    body: ...
}
```

The actual object has more functionality, but this is useful for learning.

So:

```JavaScript
response.status
```

might give:

```
200
```

And:

```JavaScript
response.ok
```

might give:

```
true
```

20. What exactly is `response.ok`?

Your mental model was correct.

`response` is an object.

`ok` is a property on that object.

```JavaScript
response.ok
```

reads that property's value.

For the Fetch API, **`ok`** is **`true`** when the HTTP status is in the **200–299 range**.

Otherwise it is **`false`**.

So:

```JavaScript
if (response.ok) {
    // HTTP status is 200–299
```

```
}
```

And:

```
JavaScript
if (!response.ok) {
    // HTTP status is outside 200–299
}
```

Important distinction

HTTP defines the meaning of the status codes.

The Fetch API provides the convenient **`ok`** property based on those status codes.

---

21. A 404 does NOT normally make `fetch()` reject

This is extremely important.

Suppose the server sends:

```
404 Not Found
```

The network request succeeded.

The browser successfully received an HTTP response.

Therefore, **`fetch()`** normally **fulfills its Promise**.

So:

```
JavaScript
fetch(url)
    .then((response) => {
        console.log(response.status);
    })
    .catch((error) => {
        console.log(error);
    });
```

will normally execute **`.then()`** and print:

```
404
```

It will **not automatically go to `.catch()` just because the HTTP status is 404 or 500.**

---

22. Why?

Because there are two different meanings of "failure":

HTTP-level failure

The server says:

```
404 Not Found
```

But communication worked.

```
Request
 ↓
Server
 ↓
404 Response
 ↓
Browser received it successfully
```

Network-level failure

The browser couldn't successfully obtain an HTTP response.

For example, certain network/connectivity failures can cause:

```
fetch()
 ↓
Promise rejected
 ↓
.catch()
```

So:

> **HTTP error ≠ Promise rejection**

---

23. `.catch()` can handle network failures

For example:

```
JavaScript
fetch(url)
    .then((response) => {
        return response.json();
    })
    .catch((error) => {
        console.log(error);
    });
```

If **`fetch()`** itself rejects because the network operation failed, **`.catch()`** can handle that rejection.

This is one important use of **`.catch()`**.

---

24. But `.catch()` can handle much more than network failures

`.catch()` can handle a rejection/error coming from earlier parts of the Promise chain.

For example:

```JavaScript
fetch(url)
    .then((response) => {
        throw new Error("Something went wrong");
    })
    .catch((error) => {
        console.log(error);
    });
```

The **`throw`** causes the Promise produced by that **`.then()`** to become rejected.

Then **`.catch()`** receives the error.

So **`.catch()`** can handle:

- a rejection from **`fetch()`**
- an error thrown inside **`.then()`**
- a rejected Promise returned from an earlier **`.then()`**
- other rejected Promises in the chain

---

25. How do we turn a 404 into something `.catch()` handles?

We manually check **`response.ok`**:

```JavaScript
fetch(url)
    .then((response) => {

        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }

        return response.json();

    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
    });
```

Suppose the server returns:

```
404
```

Then:

```JavaScript
response.ok === false
```

So:

```JavaScript
throw new Error(...)
```

executes.

That makes the Promise returned by that `.then()` rejected.

Then:

```JavaScript
.catch(...)
```

receives the error.

---

26. Yes, you CAN handle HTTP errors inside `.then()`

Your intuition was correct.

You could do:

```JavaScript
fetch(url)
    .then((response) => {

        if (response.status === 404) {
            console.log("User not found");
        }

        if (response.status === 500) {
            console.log("Server problem");
        }

    });
```

Because a 404 is still a fulfilled `fetch()` Promise.

But the common convention is often to convert HTTP failures into rejected Promises:

```JavaScript
if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
}
```

Then you have a common error-handling path:

```
Success
   ↓
.then()

Network error
   ↓
.catch()

HTTP error converted using throw
   ↓
.catch()
```

27. What is `response.json()`?

This was another important point.

When you write:

```JavaScript
.then((response) => {
    return response.json();
})
```

`response` is the **original Response object received from `fetch()`**.

You aren't creating another response.

`response.json()` reads the response body and parses it as JSON.

And importantly:

> **`response.json()` returns another Promise.**

So:

```
fetch()
  ↓
Promise
  ↓
Response object
  ↓
response.json()
  ↓
another Promise
  ↓
parsed JavaScript value
```

28. Why does the next `.then()` get `data`?

Consider:

```JavaScript
fetch(url)
    .then((response) => {
        return response.json();
    })
    .then((data) => {
        console.log(data);
    });
```

First:

```JavaScript
.then((response) => ...)
```

gets the Response object.

Then:

```JavaScript
return response.json();
```

returns another Promise.

When that Promise fulfills, its value is passed into the next **`.then()`**.

So:

```JavaScript
.then((data) => ...)
```

gets the parsed JSON.

Again, **`data`** is just a parameter name.

You could write:

```JavaScript
.then((banana) => {
    console.log(banana);
});
```

and it would work.

---

29. Why don't we manually pass arguments to `.then()`?

Normally:

```JavaScript
function greet(name) {
    console.log(name);
}

greet("John");
```

We manually pass **`"John"`**.

But **`.then()`** takes a **callback function**.

For example:

```JavaScript
promise.then((value) => {
    console.log(value);
});
```

You are giving **`.then()`** a function.

Later, the Promise machinery calls your function.

Conceptually:

```JavaScript
callback(resolvedValue);
```

So your callback's parameter receives the value.

Same with `.catch()`:

```JavaScript
.catch((error) => {
    console.log(error);
});
```

Conceptually:

```JavaScript
callback(rejectionReason);
```

Therefore:

> **You provide the callback; the Promise system calls the callback and supplies the argument.**

---

30. This is why it's called a callback

You write:

```JavaScript
.then((response) => {
    console.log(response);
});
```

You're basically saying:

> "Here's a function. Don't execute it right now. Execute it later when the Promise has a result."

The Promise machinery calls it later.

That's why `response` receives its value even though you didn't explicitly pass it yourself.

---

31. Promise chaining

Consider:

```JavaScript
fetch(url)
    .then((response) => {
        return response.json();
    })
    .then((data) => {
        console.log(data);
```

```
    })
    .catch((error) => {
        console.log(error);
    });
```

Think:

```
fetch()
  ↓
Promise
  ↓
fulfilled with Response
  ↓
.then(response)
  ↓
response.json()
  ↓
Promise
  ↓
fulfilled with parsed JSON
  ↓
.then(data)
  ↓
data
```

And if something goes wrong:

```
Something rejects/throws
        ↓
    .catch(error)
```

32. The production side of Promises

This was your final major question.

Suppose you're building an API using Node.js.

The server might internally use a Promise for a database operation:

```
JavaScript
function getUserFromDatabase(id) {

    return new Promise((resolve, reject) => {

        database.query(
            "SELECT * FROM users WHERE id = ?",
            [id],
            (error, result) => {

                if (error) {
                    reject(error);
                } else {
                    resolve(result);
                }

            }
        );

    });
}
```

Here:

```JavaScript
resolve(result);
```

means:

> "The database operation succeeded; give this result to whoever is waiting for this Promise."

And:

```JavaScript
reject(error);
```

means:

> "The database operation failed."

---

33. Then the API server uses that Promise

For example:

```JavaScript
app.get("/users/:id", async (req, res) => {

    try {

        const user = await getUserFromDatabase(req.params.id);

        res.status(200).json({
            data: user
        });

    } catch (error) {

        res.status(500).json({
            message: "Database failed"
        });

    }

});
```

Here:

```JavaScript
await getUserFromDatabase(...)
```

waits for the Promise.

If the database operation succeeds:

```
database
   ↓
```

```
resolve(result)
   ↓
Promise fulfilled
   ↓
await gets result
   ↓
server sends HTTP 200
```

If the database operation fails:

```
database
   ↓
reject(error)
   ↓
Promise rejected
   ↓
catch receives error
   ↓
server sends HTTP 500
```

34. `resolve/reject` are NOT sent to the frontend

This is extremely important.

Suppose the backend does:

```
JavaScript
resolve(user);
```

The browser does **not** receive:

```
resolve(user)
```

over the network.

Instead:

```
resolve(user)
     ↓
internal server-side Promise
     ↓
server gets user
     ↓
server creates HTTP response
     ↓
HTTP 200 + JSON
     ↓
network
     ↓
frontend
```

The Promise exists inside the server's JavaScript environment.

---

35. Promises don't bundle HTTP metadata automatically

`resolve()` doesn't automatically know about:

- HTTP status
- HTTP headers
- JSON
- request URL
- etc.

You decide what value to give it.

For example:

```javascript
resolve(result);
```

or:

```javascript
resolve({
    data: result,
    statusCode: 200
});
```

or:

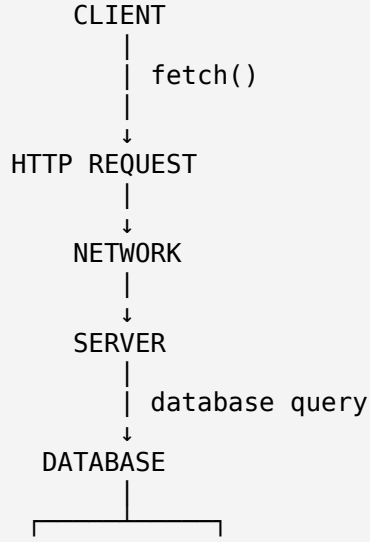
```javascript
resolve({
    user: result,
    message: "User found"
});
```

Whatever you pass becomes the Promise's fulfilled value.

---

36. The complete client → server → database picture

This is probably the most useful diagram to remember:



```
                ↓           ↓
            success      failure
                |           |
            resolve()    reject()
                |           |
                ↓           ↓
            Promise      Promise
            fulfilled    rejected
                |           |
                ↓           ↓
              await       catch
                |_____   ___|
                      |
                      ↓
                SERVER DECIDES
                HTTP RESPONSE
                      |
                ┌─────┴─────┐
                ↓           ↓
             200           404/500
                |_____   ___|
                      |
                      ↓
                   NETWORK
                      ↓
                   CLIENT
                      |
                      ↓
                   fetch()
                      |
            Promise fulfilled
            if HTTP response
                      |
                      ↓
                   .then()
```

And if the network itself fails on the client side:

```
fetch()
  ↓
network failure
  ↓
fetch Promise rejects
  ↓
.catch()
```

37. The most important distinction of everything we learned

Keep these two concepts separate:

HTTP

Communication **between client and server**:

```
Request → Server → Response
```

HTTP defines:

```
status codes
headers
body
methods
```

```
etc.
```

Promise

A JavaScript mechanism for representing an **eventual result of asynchronous work**:

```
Pending
   ↓
resolve → Fulfilled
   ↓
.then()

or

reject → Rejected
   ↓
.catch()
```

They work together, but they aren't the same thing.

---

38. Final mental model

If you remember only this, you're in a very good position:

```
              PROMISE PRODUCER
                     │
          ┌──────────┴──────────┐
          ↓                     ↓
     resolve()              reject()
          ↓                     ↓
     Fulfilled              Rejected
          ↓                     ↓
      .then()               .catch()
```

With **`fetch()`**:

```
                fetch()
                   │
           creates/returns
                   ↓
                Promise
                   │
          ┌────────┴────────┐
          ↓                 ↓
  HTTP response received   Network failure
          ↓                    ↓
     fulfilled             rejected
          ↓                    ↓
      .then()              .catch()
```

But:

```
404 / 500
```

are **HTTP errors**, not automatically Promise rejections from **`fetch()`**.

So:

```
404
 ↓
fetch receives valid HTTP response
 ↓
Promise fulfilled
 ↓
.then(response)
 ↓
response.ok === false
```

If you want to make that a Promise rejection:

```javascript
if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
}
```

Then:

```
throw
 ↓
rejected Promise
 ↓
.catch()
```

And finally, on the **API/server side**:

```
Database operation
      ↓
Promise
  ┌─────┴─────┐
  ↓           ↓
resolve   reject
  ↓           ↓
success   failure
  └─────┬─────┘
      ↓
Server handles result
      ↓
Creates HTTP response
      ↓
200 / 404 / 500 etc.
      ↓
sent to client
```

In one sentence:

> **`resolve()` and `reject()` are tools used by whoever PRODUCES a Promise; `.then()` and `.catch()` are tools used by whoever CONSUMES it. `fetch()` is a Promise-producing API, HTTP is the communication protocol between client and server, and the server may itself use Promises internally for things like database/network operations — but those Promises never travel across the network.**